

Instruction Economics in Agent System Prompts

Attributing Cost and Behavioural Return to Individual Directives, with a Released Tool That Does It for Your Prompt

Jashwanth Lakshmi Sai Yenugu
Independent Researcher
yenugu.jashwanth@gmail.com
<https://orcid.org/0009-0008-1831-2615>

September 9, 2026

Abstract

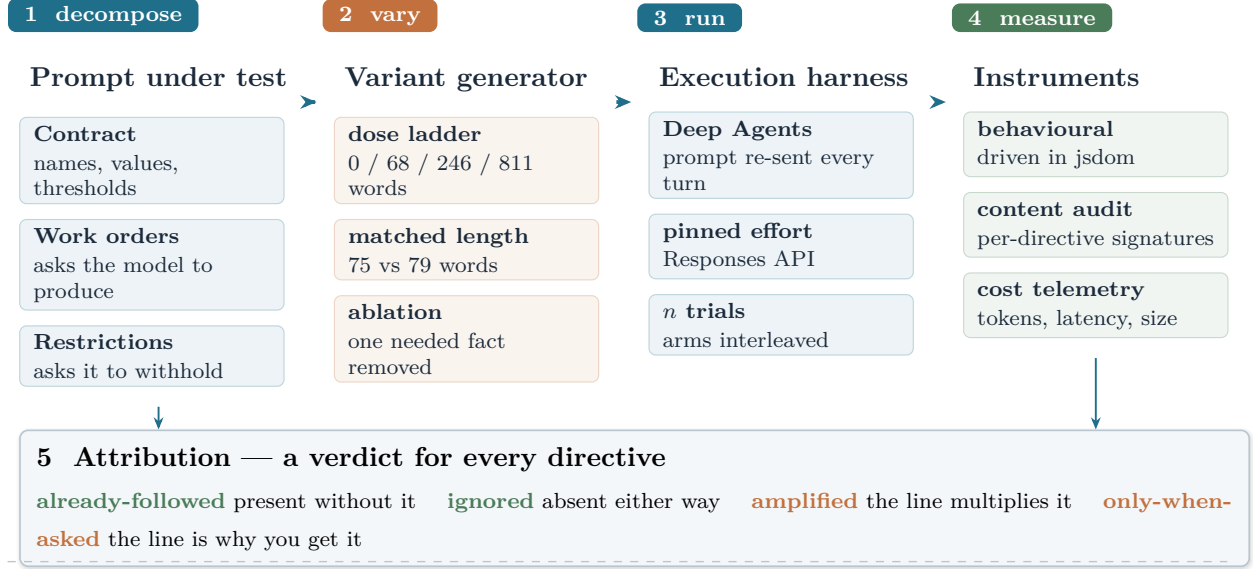
Agent system prompts — the persistent instruction files that drive coding agents, whether written as a vendor system prompt, a packaged skill, a `CLAUDE.md` or an `AGENTS.md` — accumulate general engineering guidance alongside the project-specific facts a model cannot infer. The guidance is correct, and a frontier model can recite all of it. Practitioners disagree about what it does: one camp treats it as harmless insurance, the other as bloat that degrades the model. Neither position is usually measured. We measure it, on a pipeline that observes every generated artifact with three independent instruments — a behavioural grader that drives it like a user, a content audit that counts per-directive compliance signatures, and cost telemetry.

Across 198 runs on two OpenAI GPT-5.6 models under pinned reasoning effort, we append 0 to 811 words of such guidance to otherwise fixed build tasks and hold everything else constant. Three results follow. **It costs.** Output tokens rise 42% and latency 38% at 811 words, in every one of twelve paired cells ($p = 0.00024$), and on an open-ended task 811 words doubled latency. **It buys exactly what it asks for.** The model does not apply the guidance unprompted: a reduced-motion query appears in 0 of 12 unpadded files and 12 of 12 padded ones, and **rem** units rise 5.4-fold per unit of output. On the open-ended task the model writes six times more JavaScript. Generic guidance is not a reminder the model already honours; it is a set of feature requests the model fills and bills for. **Correctness does not move.** Behavioural correctness and rule-following hold at ceiling across three increasingly demanding instruments, and a positive control confirms the pipeline registers a 26-point drop when a single needed fact is removed. Cost is not a correctness trade in either direction.

Word count is the wrong variable. At matched length, 75 words asking for work cost +22% output while 79 words asking for restraint saved 31% — a 53-point swing, both arms fully correct. A line is billed for what it asks the model to do. **We release the method as a tool, so the result is usable rather than merely reported.** The paper's practical output is a decision procedure for the two questions every prompt owner has — *what must stay in, and what can come out* — each answered by measurement on the reader's own model rather than by taste. It ships as an installable agent skill any agent can execute: point it at your prompt and a command that runs your real task, and it classifies every line, runs the task without the generic guidance, attributes a verdict to each directive, proposes a trim of dead weight only, measures the original against the trimmed prompt, and reports what was removed, what was deliberately kept and which feature each kept line buys, and the measured token and latency delta. It never deletes: lines surviving attribution are features being bought, and that is the owner's call.

We also contribute a three-control validation protocol for behavioural graders — a correct reference must score 100%, an empty artifact 0%, and a single-rule mutation must fail exactly its own checks — with six defects it catches, each of which had produced a clean and wrong number in this study.

A The study how every result in this paper was produced



B The tool `prompt-contract`, released — the same pipeline, run on *your* prompt

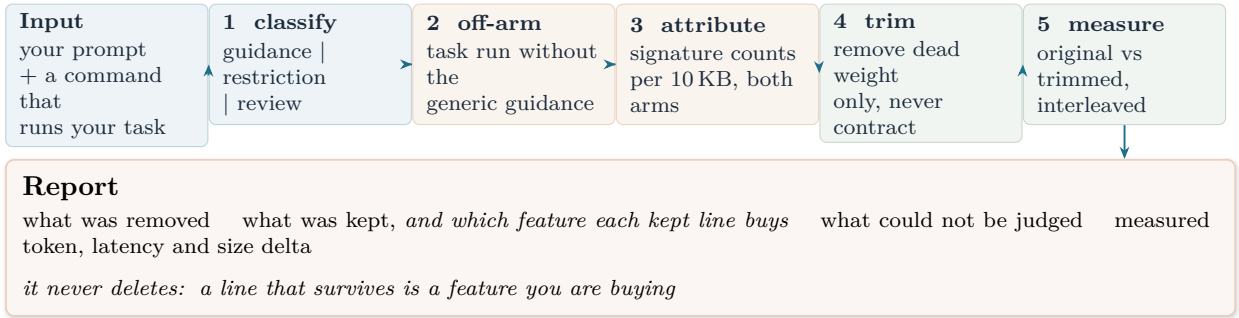


Figure 1: **A** The study. A prompt is decomposed into contract, work orders and restrictions; the contract is never ablated. A generator emits variants differing in one respect only, every variant runs through the same harness at pinned reasoning effort with arms interleaved, and three independent instruments observe each artifact — whether it works, what it contains, what it cost. Their differences across arms yield a per-directive verdict. **B** The tool. The same pipeline, packaged as an installable skill that runs it against the reader’s own prompt, model and task, and reports what to remove, what to keep, and what the removal is worth.

1 Introduction

An agent system prompt is a persistent instruction file interposed between a user and a model on every turn. The form varies — a vendor system prompt, a packaged *skill*, a repository-level `CLAUDE.md` or `AGENTS.md`, a `.cursorrules` — but the function is constant: text the model receives before and alongside each request, maintained by a team, and growing over time as people add rules in response to things that went wrong.

Such files accumulate two kinds of content that are rarely distinguished. The first is **project-specific**: component names, output schemas, thresholds, the precedence between conflicting rules,

invariants of the domain. No model can infer these, and a model without them will guess plausibly and wrongly. The second is **general engineering guidance**: write semantic markup, order your headings, label your controls, name your constants, fail gracefully. Any current frontier model can give a correct account of all of it on request.

This paper is about what that second kind of content does. Two folk positions are common and opposed. **Insurance**: the guidance is redundant but harmless — the model already knows it, restating it costs a few tokens and guards against regressions. **Bloat**: the guidance dilutes attention across a longer context and the work is worse for it. Both are testable, and both turn out to be wrong in instructive ways.

We measure three things:

- **Cost**. What does the guidance add to tokens and latency, and on which side of the bill?
- **Return**. Does the model's output change, and if so, where exactly?
- **Correctness**. Does anything the task requires get worse?

The answers are: a lot, and on the output side where caching cannot help; yes, in precisely the places the guidance names; and no, on instruments that demonstrably detect a real loss.

Scope, and what is instantiated. The claims are about agent system prompts in general — nothing in the method depends on the domain, the vendor or the file format. What the experiments instantiate is necessarily narrower, and we state it plainly so that transfer remains a judgement the reader makes rather than one we assume.

dimension	instantiated as
models	<code>gpt-5.6-luna</code> , <code>gpt-5.6-terra</code> (OpenAI); all runs 6–9 September 2026
harness	LangChain Deep Agents, filesystem backend, reasoning effort pinned to <code>medium</code>
task	self-contained front-end artifacts: a quiz, a data table, a pricing engine, a validation form,
family	a dashboard
grading	behavioural execution in <code>jsdom</code> , plus mechanical content signatures
prompt scale	0–811 words of guidance on the ladder; 1,094 vs 5,840 words at skill scale

What this buys the reader. The practical result of the work is a decision procedure for the two questions every prompt owner actually has — *what must go in*, and *what can come out* — with each answer backed by measurement rather than by taste:

- **What must go in.** Everything the model cannot infer (§2), plus any directive the measurement shows is the sole reason a behaviour appears at all. A reduced-motion query appeared in 0 of 12 artifacts without its line and 12 of 12 with it: that line is not overhead, it is the feature.
- **What can come out.** Directives the model already satisfies unprompted, and directives it ignores however firmly they are stated. In the ladder both categories are populated, and neither is identifiable by reading.
- **What it is worth.** The removal is measured, not asserted: output tokens, latency and artifact size, original against trimmed, with intervals.
- **What it costs to find out.** One run of your own task without the generic guidance — roughly \$0.15 for a single-artifact task.

§9 packages exactly this as an installable skill, so the procedure runs against the reader's own prompt and model rather than requiring them to trust ours.

The taxonomy of §2, the attribution procedure of §3, the tool of §9 and the grader-validation protocol of §10 are domain-independent, and are what we expect to transfer. The specific bucket assignments of §6 — that reduced-motion is adopted only when asked, that comments are ignored either way — are properties of these models on these tasks at this date. The released tool re-derives them on the reader's own model rather than shipping them as fact.

2 What a prompt is made of

The measurements rest on a three-way partition of the lines in a prompt. It is worth stating precisely, because the partition is what makes the results interpretable and because two of its categories behave in opposite directions.

Contract. A line stating something the model cannot infer from the task: an exact name, an exact value, a structural requirement, a precedence rule, a domain invariant. A contract line carries information with no support in the model's prior — the space of plausible completions is wide and only one point in it is correct. Contract is never a deletion candidate, and in every experiment below it is preserved verbatim across arms. It is removed in exactly one place, the positive control of §7, where the point is to prove the pipeline can detect a loss.

Work order. A line asking the model to produce something it would not otherwise produce: "handle the empty case", "emit structured logs", "add a loading state". Grammatically an imperative with a direct object that expands the artifact. A work order is a purchase — it enlarges the output and is billed accordingly. It is legitimate content; the point is that it is a purchase rather than a reminder.

Restriction. A line asking the model to withhold: "never use X", "do not add commentary", "no speculative abstraction". An imperative that contracts the space of acceptable outputs. §8 shows these do not merely avoid the bill; at matched length they reduce it.

The partition is not always crisp — "prefer `const` to `var`" both requests and restricts — and the released tool reports an explicit *review* category for lines it cannot confidently place rather than guessing. That conservatism matters: a misclassified contract line is the one error whose consequence is irreversible.

The central empirical fact is that membership does not determine behaviour. A work order may be one the model already fills unprompted, in which case the line changes nothing. A line's *effect* is therefore a fourth property, orthogonal to its grammar, and observable only by counterfactual: run the task without it and see what changes. That observation is the method of this paper and the product of the tool it releases.

3 Apparatus

Pipeline. Every experiment is one traversal of Figure 1 (panel A). A prompt is decomposed as in §2; a generator emits variants differing in exactly one respect; each variant executes n times through an identical harness; three instruments observe every artifact; and differences between arms are attributed to the directive that distinguishes them.

Instruments. Three, deliberately independent, because a single instrument that fails silently produces a clean and wrong result:

1. *Behavioural grader.* The artifact is loaded in jsdom and driven as a user would drive it — clicking sort controls, typing into filters, paging, triggering exports — and scored on whether each required behaviour occurs. This answers *does it work?* and nothing else. Grader construction is itself validated by the protocol of §10.

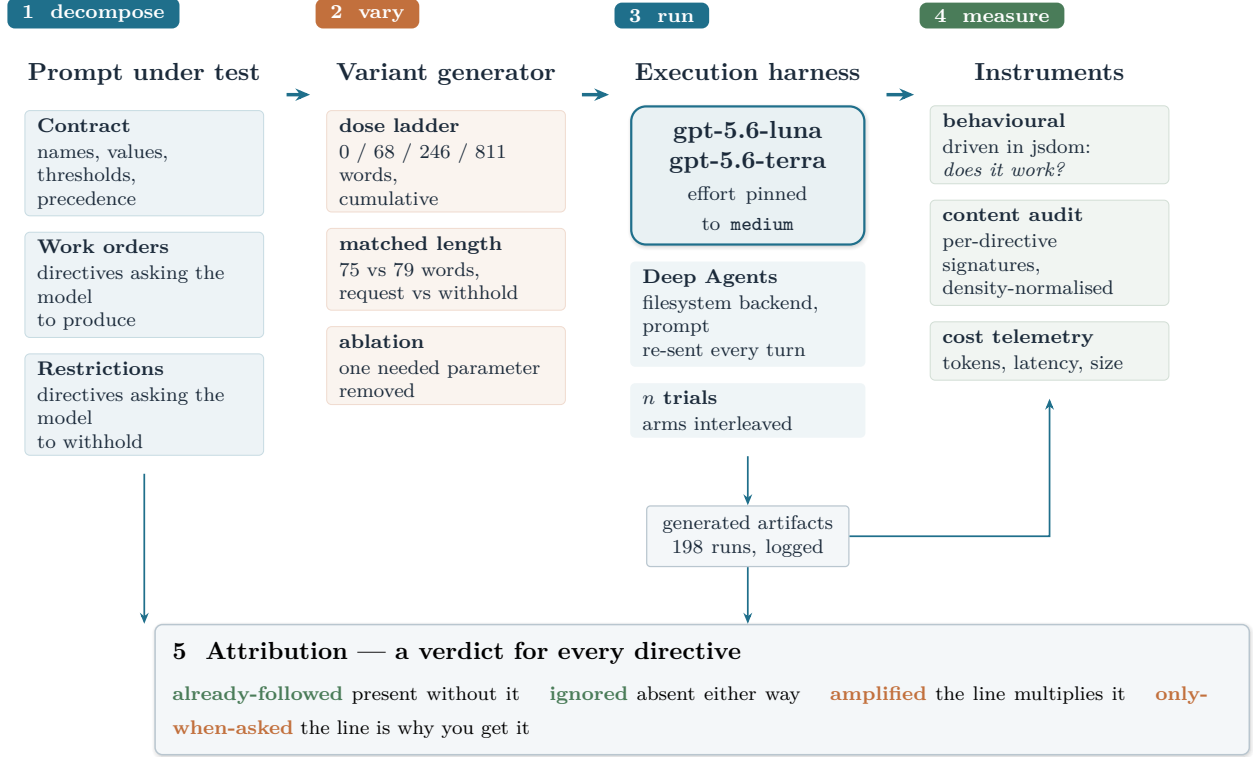


Figure 2: The apparatus. A prompt is decomposed into contract, work orders and restrictions; the contract is never ablated. A generator emits variants that differ in one respect only. Every variant runs through the same harness, at the same pinned reasoning effort, with arms interleaved so that drift in API response time during a session lands on all of them equally. Three independent instruments then observe each artifact — whether it works, what it contains, and what it cost — and their differences across arms yield a per-directive verdict. Each results section is one traversal of this pipeline.

2. *Content audit.* For each directive, a mechanical signature — a regular expression over the artifact — whose count indicates compliance. Signatures need not be individually accurate: the audit differences two arms with the same signature, so systematic bias applies to both and cancels. Counts are normalised per 10 KB, because a longer artifact contains more of everything.
3. *Cost telemetry.* Output tokens as reported by the API (inclusive of reasoning tokens), wall-clock latency, and artifact size in bytes. Size accompanies tokens as the visible-output measure, since token counts alone cannot separate a longer artifact from longer reasoning.

Statistics. Effects are reported as means with 95% bootstrap intervals (10,000 resamples, seeded). Where a design admits pairing we compare within each task×model×trial cell rather than across pooled means, and report the count of pairs moving in each direction alongside a one-sided sign test. We give both trial-level and cell-level granularity wherever they differ, because trial-index pairing is nominal rather than matched. Permutation tests in §8 are exact, enumerating all all 924 relabellings rather than sampling.

Pre-registration. Each experiment's design, admission criteria and success conditions were fixed in code before its runs. We do not claim external registration; the commit history is the only attestation, and we say so rather than implying more. Where a pre-registered criterion failed, it is

reported as failed — see §8.

4 Design

Guidance ladder. Four cumulative levels of general web-development guidance, appended after an otherwise fixed task and fixed in code before any run. Every statement is correct, generic, and free of task-specific fact; none of it names anything a task requires, and none of it is wrong.

level	words	adds
L0	0	nothing
L1	68	semantic elements, heading order, form labels, contrast, keyboard reachability, focus indicators
L2	246	CSS conventions (flat specificity, custom properties, <code>rem</code> , flex/grid, responsive) and JavaScript conventions (<code>const/let</code> , <code>addEventListener</code> , small functions, named constants)
L3	811	performance, maintainability and comments, accessibility beyond the basics (<code>aria-live</code> , tab order, reduced-motion), robustness and empty states, state management, error handling, user feedback

The ladder is cumulative, so level is confounded with length by construction. That confound is the subject of §8, which separates the two directly.

Tasks. Two self-contained builds carry the ladder: a five-question quiz with gated submission and full retry, and a sortable, filterable employee table with pagination and CSV export. Later experiments add a five-rule compliance task, three deliberately hard calibration tasks, a pricing-engine positive control, an open-ended dashboard brief, and a skill-scale arm.

Models and harness. `gpt-5.6-luna` and `gpt-5.6-terra` (all runs 6–9 September 2026), reasoning effort pinned to `medium` through the Responses API with text verbosity at its default, executed through LangChain Deep Agents (0.6.12) with a filesystem backend. Effort is pinned because provider-default effort differs between models and materially changes what a model produces; leaving it floating would confound every comparison between prompts. The harness matters twice over: it prepends its own base prompt and tool schemas to every call, so the zero-guidance arm is zero *added* guidance rather than an empty system prompt; and it re-sends the system prompt on each turn of the agent loop, the condition under which prompt content should be most expensive.

Scale and spend. 198 runs, \$18.12 estimated from list prices (\$1.25 per million input tokens, \$10 per million output), not billed: a 16-run direct-API pilot, a 20-run Deep Agents replication, a 48-run guidance ladder, a 24-run compliance test, a 12-run hard-task calibration, a 12-run positive control, a 36-run diversity experiment, an 18-run test of what a line asks for, and a 12-run arm at skill scale. API nondeterminism means runs are not seed-reproducible; the complete logs are released so every number can be re-derived offline, and `experiments/verify_paper.py` does exactly that, exiting non-zero on any mismatch.

5 What generic guidance costs

Pooled across both tasks and both models, $n = 12$ per level:

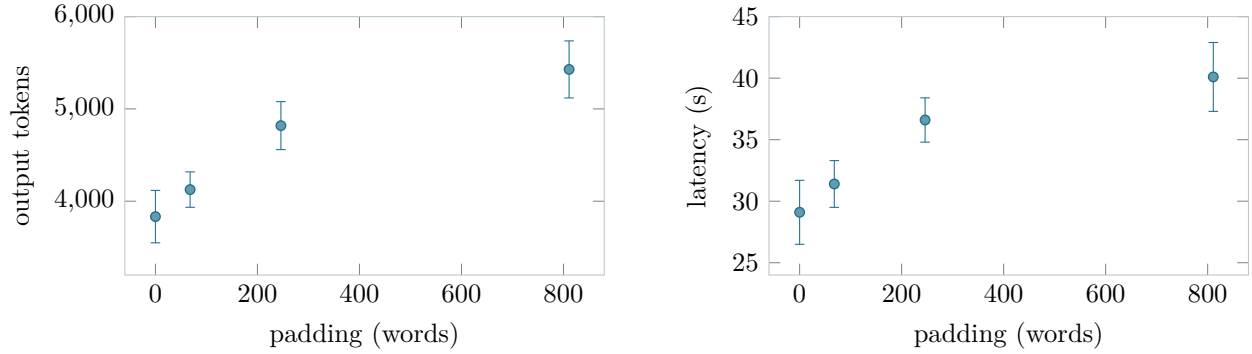


Figure 3: Cost rises monotonically with padding that adds no information. Output tokens (left) and end-to-end latency (right) against the quantity of general-knowledge padding, pooled over two tasks and two models, $n = 12$ per dose. Bars are 95% bootstrap intervals. The padding is *input*, yet it is the output that grows by 42%. Comparing within each task/model/trial cell, the increase holds in all twelve pairs (sign test $p = 0.00024$).

level	words	output tokens	latency	file size
L0	0	3,833	29.1s	12.1k
L1	68	4,126 (+8%)	31.4s (+8%)	
L2	246	4,819 (+26%)	36.6s (+26%)	
L3	811	5,428 (+42%)	40.1s (+38%)	18.2k (+50%)

Figure 3 plots both. The relationship is monotonic in both columns, and the endpoints are well separated: 95% bootstrap intervals put L0 output at 3,833 [3,549, 4,105] tokens against L3's 5,428 [5,119, 5,701], and latency at 29.1 [26.6, 31.6] seconds against 40.1 [37.3, 42.9].

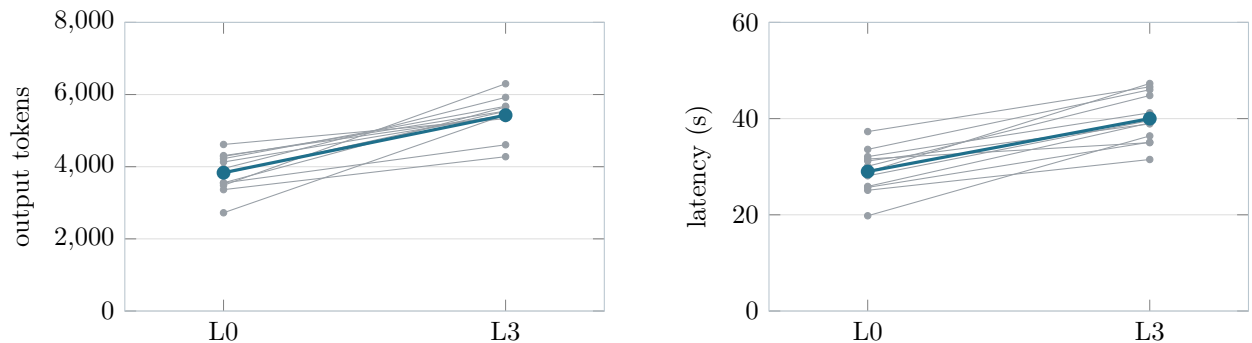


Figure 4: Every pair rises. Each grey line is one task, model and trial at zero guidance (L0) and 811 words (L3); the blue line is the mean. Output tokens rise in 12 of 12 pairs and latency in 12 of 12, one-sided sign test $p = 0.00024$ for both.

Every pair rises. Comparing L0 to L3 within each task, model and trial rather than across pooled means, the increase holds in all twelve pairs (Figure 4): output +1,595 [+1,241, +1,989] tokens and latency +11.0 [+8.7, +13.3] seconds, one-sided sign test $p = 0.00024$ for both. That test treats the twelve trial-index pairings as independent; at the coarser granularity of the four task×model cells, all four of which rise, the exact one-sided value is $p = 0.0625$. We report both and rest the claim on the direction, which is unanimous at either granularity. Output tokens explain most of the latency variance, and file size grows 50%, so the growth is visible output rather than hidden reasoning.

Most of the bill arrives at the output. Both sides rise, and in proportional terms the input side rises by more: input tokens go from 10,422 to 15,194 (+46%, since the harness re-sends the prompt on every turn) against output's +42%. The two are not priced alike, however. At the list

not replicate there. Output tokens rose 4% and latency 3%; the increase appears in only 4 of 6 paired cells and the two models disagree (+11% and −1%). Behavioural scores were 100% for the contracts arm and 98.3% for the full one.

The composition explains it, in the paper's own terms. The 4,746 added words are roughly a quarter code templates and two thirds descriptive specification — state machines, event names, pitfalls — with only about 8% resembling the ladder's generic guidance. Templates *show* and specifications *describe*; neither asks the model to produce anything it was not already going to produce. The ladder's 811 words are directives that ask for work end to end, and they cost 42%. **Six times the words, a tenth of the cost.** Length is not the variable.

Latency and run order. The ladder runner executed levels in order within each cell, so latency could absorb any drift in API response time during a session; the later diversity runner interleaves arms for this reason. Output tokens are immune to drift and carry the same finding, and we treat them as the primary cost measure.

6 What it buys

If the model already applied the guidance unprompted, its output would not change when the guidance was added. It does — and in exactly the places the guidance names. A content audit of the same files (`experiments/audit/content.py`) counts, per file, the constructs each directive asks for.

directive in the guidance	L0, per file	L3, per file	open-ended task, minimal → padded
respect	0 of 12 files	12 of 12 files	0 → 12 of 12
prefers-reduced-motion			
ARIA attributes	7.3	14.1 (×1.3 per KB)	2.5 → 25.1
rem units	4.8	38.5 (×5.4 per KB)	0.0 → 81.2
CSS custom properties	7.9	18.8 (×1.6 per KB)	12.3 → 25.4
semantic elements	3.7	7.3 (×1.3 per KB)	8.8 → 14.7
const/let declarations	22.5	36.5	7.3 → 39.3
addEventListener	4.5	4.6	0.6 → 4.2
inline onclick , var	0	0	0 → 0
try/catch , code comments	0	0	0 → 0

Padded files are simply larger, so raw counts overstate adoption; the density column above is the honest one. On that measure **rem** units are the clearest genuine adoption (×5.4), custom properties and ARIA rise modestly, and reduced-motion goes from wholly absent to universal. Three patterns appear. Some guidance was **already followed** at zero padding: no inline handlers, no **var**, listeners attached properly. Some was **adopted only when asked**: reduced-motion support went from absent to universal, ARIA doubled, **rem** units rose 5.4-fold by density, custom properties more than doubled. Some was **ignored either way**: the guidance asks for comments and graceful error handling, and no file at any level contains a comment or a **catch**. On the open-ended dashboard, where the brief left the most room, the model wrote six times more JavaScript under guidance (1,272 → 7,939 characters) while markup stayed flat: it built interaction and accessibility scaffolding it otherwise omitted.

This is the mechanism behind §5. The guidance is not a reminder of behaviour the model already exhibits; it is a checklist the model works through. Each item it adopts is a feature the output did not have before, and each is paid for in tokens. Whether that is a return or a cost depends entirely on whether the team wanted the feature. A team that cuts accessibility guidance on the strength of "the model knows this" will lose the ARIA it was getting.

7 Correctness holds at ceiling

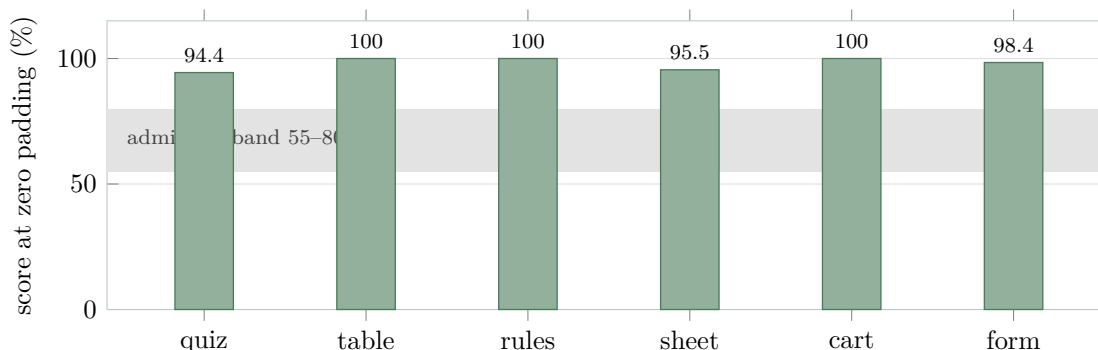


Figure 5: Every instrument sat at ceiling. Zero-padding scores for the six task instruments across three attempts: the two dose-ladder builds, the five-rule compliance test, and the three deliberately hard calibration tasks. The shaded band is the pre-registered 55–80% admission range that a task had to fall into before being carried to the ladder. The lowest score is 94.4%; nothing entered the band, and the band was not moved.

The cost is not a correctness trade. We tested for degradation with three instruments of increasing sensitivity; correctness held at ceiling on every one.

Behavioural correctness. Of the 48 ladder outputs, 47 pass every check; the single defect — a `RadioNodeList` has no `.closest()` — occurred at zero guidance. Both task families sit at ceiling.

Rule-following. Five mechanically checkable house rules were stated once, before the task, with the ladder appended after them. The rules were kept at every level: one file at L1 placed a hex colour outside `:root`, one carried an unprefixed id, and nothing else was ever broken. Three of the five rules were never broken under any condition.

Deliberately hard tasks. Because both instruments were at ceiling, we wrote three tasks to be difficult — a formula engine with cycle detection and undo, a pricing engine with non-stacking coupon precedence, a form with exact cross-field error strings — and calibrated them at zero guidance against an admission band of 55–80% fixed in advance. They scored 95.5%, 100% and 98.4% across 22, 80 and 31 checks. Nothing entered the band, and the band was not moved. Every instrument sat at ceiling (Figure 5).

The pipeline is sensitive. A ceiling result means something only if the instrument can move, so we ran a positive control: the pricing task, the same grader, the same model and effort, with one parameter the model cannot infer removed while the requirement to display the value is kept. Success was defined in advance as any ablated arm below 80%.

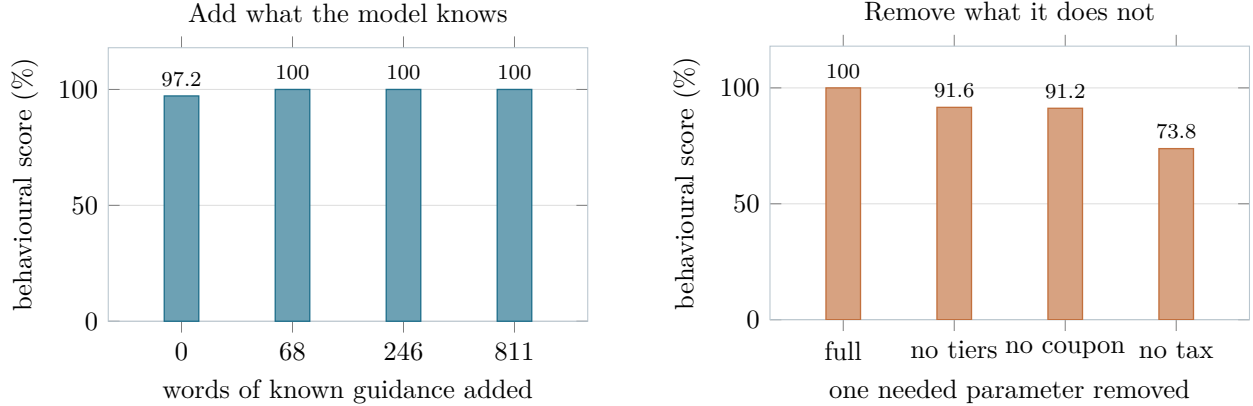


Figure 6: Sensitivity. Left: adding up to 811 words of guidance leaves the ladder tasks at ceiling (97.2% to 100.0%, $n = 12$ per bar). Right: the positive control on the pricing task, where removing a single parameter the model cannot infer drops the score by 26.2 points ($n = 4$ per bar), failing only the checks that depend on it. The pipeline registers a large, targeted loss of information; its power against a small diffuse effect is bounded in §7.

arm	removed	score
full (control)	—	100.0%
minus tiers	the 5/10/15% thresholds	91.6%
minus coupon	the non-stacking precedence rules	91.2%
minus tax	the 8.25% rate	73.8%

	subtotal	discount	shipping	tax	total
full	100%	100%	100%	100%	100%
no tiers	100%	85%	100%	85%	85%
no coupon	100%	87%	100%	87%	87%
no tax	100%	100%	100%	30%	30%

Figure 7: Where the failures land. Pass rate of the positive control’s five check families under each ablation ($n = 4$ runs per arm, 14 checks per family per run). Removing the tax rate breaks only tax and total; subtotal, discount and shipping stay at 100%. Removing the tiers or the coupon rules breaks the discount-dependent families; the coupon arm additionally fails a case-insensitivity check that lies outside these five families and so is not visible here.

Where the failures land. In the minus-tax arm only tax and total checks failed — 14 of the 15 checks in each family, in three of four runs; the exception is the empty-cart scenario, whose tax and total are zero either way — while every subtotal, discount and shipping check still passed. (Figure 6) (Figure 7) The grader isolates the removed fact and nothing else. Models sometimes guessed conventional parameters (5/10/15% tiers in two runs of four; 8.25% tax once), so conventional facts are partly recoverable from priors and arbitrary ones are not.

What this licenses. The pipeline registers a large, targeted loss of information as a 26-point drop. It does not establish power against a small, diffuse effect: with at most one failure in twelve runs per level, a per-run failure rate below roughly 25% would not have been detected. The claim is therefore bounded and stated as such — adding up to 811 words of guidance produced no degradation detectable by instruments that demonstrably detect the removal of needed information.

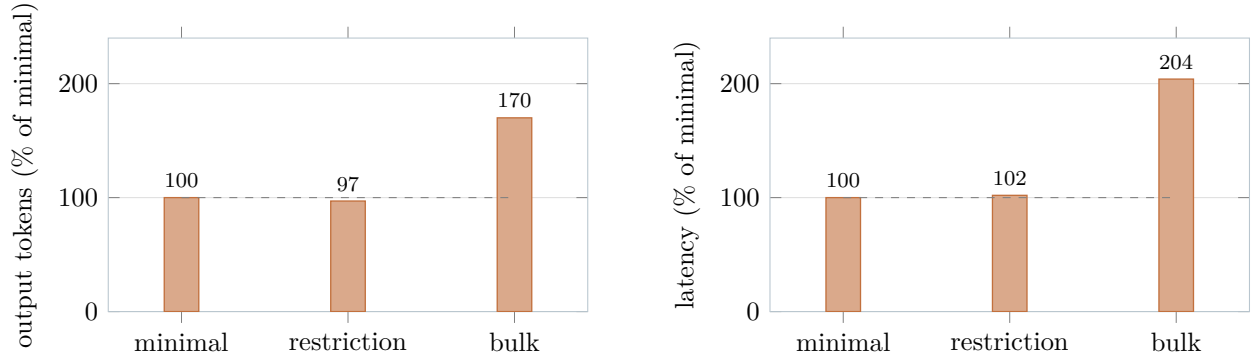


Figure 8: Bulk costs; restriction does not. Open-ended dashboard brief, $n = 12$ per bar over two models. Ninety words of restrictive rules change output tokens by -3% and latency by +2%. Eight hundred and eleven words of generic engineering guidance – change them by +70% and +104%.

Cost split by kind of line. Restriction cost nothing; bulk doubled latency (Figure 8). **What a line asks for, not how long it is.** An 18-run experiment settles this on one task text used verbatim in every arm, with the arms interleaved. Against the bare task we add 75 words of house rules that ask for *work* — a `data-testid` on every control, prefixed ids, colours centralised in `:root` — and, separately, 79 words of restrictions that ask for *less*: add no unrequested feature, no comments, no animation, no speculative abstraction.

arm	added	output tokens	latency	artifact size	behavioural score
bare	—	4,253	35.2s	13.2k	96.7%
rules (ask for work)	75 words	5,170 (+22%)	41.1s (+17%)	15.4k (+17%)	100%
restraint (ask for less)	79 words	2,949 (−31%)	23.7s (−33%)	8.9k (−32%)	100%

At matched length the two arms move in opposite directions, 53 points apart on output. The rules arm rose in 6 of 6 paired cells and the restraint arm fell in 6 of 6; the split holds on each model separately (+19%/−27% and +24%/−34%). Both scored 100% on the behavioural checks, so the cheaper arm is cheaper because it writes less, not because it works less.

This overshot our own prediction, and the pre-registered criterion therefore failed as written. We registered that the claim would hold if the rules arm exceeded bare by more than 10% and the restraint arm stayed within $\pm 10\%$ — we expected restriction to be free. It is not free; it is a 31% saving. The direction was right and the magnitude wrong, so the criterion is recorded as failed and the result reported as measured. Restriction does not merely avoid the bill, it reduces it.

Word count is therefore the wrong variable. A line is billed for what it asks the model to do, and asking for less pays.

Did the outputs converge? We also asked whether bulk or restriction makes repeated runs of one brief more alike, on a similarity metric fixed and validated before the runs. The test was inconclusive: on the primary metric the two models moved in opposite directions under both bulk and restriction (exact permutation p from .06 to .98), and the metric's own validation arm did not separate from baseline. **What the runs looked like.** On the model that started with the most range, six uninstructed runs chose five distinct accent colours and two typefaces; under either bulk or restriction they chose two colours and one face. The other model leaned blue from the start and became *less* alike under both arms — on all three components of the metric under bulk, and on two of three under restriction, the third unchanged. This is a descriptive pattern at $n = 6$ and is reported as a lead, not a result (Figure 9); a length-invariant convergence metric with its own positive-control arm is the design that would settle it.

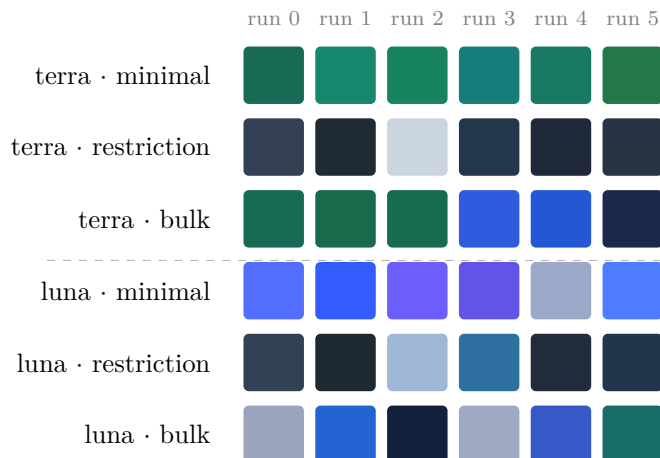


Figure 9: The dominant accent colour each run chose, extracted from its CSS. Six uninstructed runs of terra (top) span 5 distinct accents; under ninety words of restriction or 811 words of bulk they narrow to 2 and 2. Luna (bottom) leans blue from the start (3 / 2 / 3) and has little range to lose. A secondary measure at $n = 6$: a lead, not a result.

1. **Classifies** every line as guidance, restriction, or *review*. Lines it cannot confidently place are returned as review and never proposed for deletion. Run against a long production skill during development, the classifier placed the large majority of prose lines in that category, which is the correct answer for a file that is mostly contract and a demonstration that the tool declines to act where it has no signal. That artifact is not part of the released record, so the figure is reported as an observation rather than a result.
2. **Runs the off-arm**: the same task with the generic guidance removed and the contract intact, n trials.
3. **Attributes** a verdict to each directive by counting its signature per 10 KB in both arms.
4. **Proposes a trim** containing only already-followed and ignored lines.
5. **Measures** original against trimmed, interleaved, with bootstrapped intervals on output tokens, latency and artifact size.
6. **Reports** what was removed; what was deliberately kept, naming the feature each kept line buys so the owner can overrule the tool; what could not be judged; the measured cost delta; and an explicit statement that none of this establishes the output is still correct.

Two design commitments separate a useful tool from a harmful one. **It never deletes.** Lines surviving attribution are features being bought, and deciding whether to keep buying them is a product question, not a prompt question. A tool that silently strips a prompt to shorten it is a tool that silently removes a team's accessibility. **It ships a procedure, not a list.** The measured bucket assignments are included only as priors indicating which lines to check first, stamped with model and date, with an instruction to disregard them when the stamp is stale or the model differs.

The cheapest useful invocation needs only the off-arm: deleting lines the model already satisfies is the unconditional win and requires no second run. At this study's mean of \$0.09 per run, the off-arm alone is roughly \$0.28 and the full procedure — off-arm plus original-versus-trimmed — about \$0.83 for a single-artifact task; a long agentic loop costs proportionally more. The measurement component is provider-agnostic, invoking a command supplied by the user rather than an SDK, so it applies to any model, harness or language.

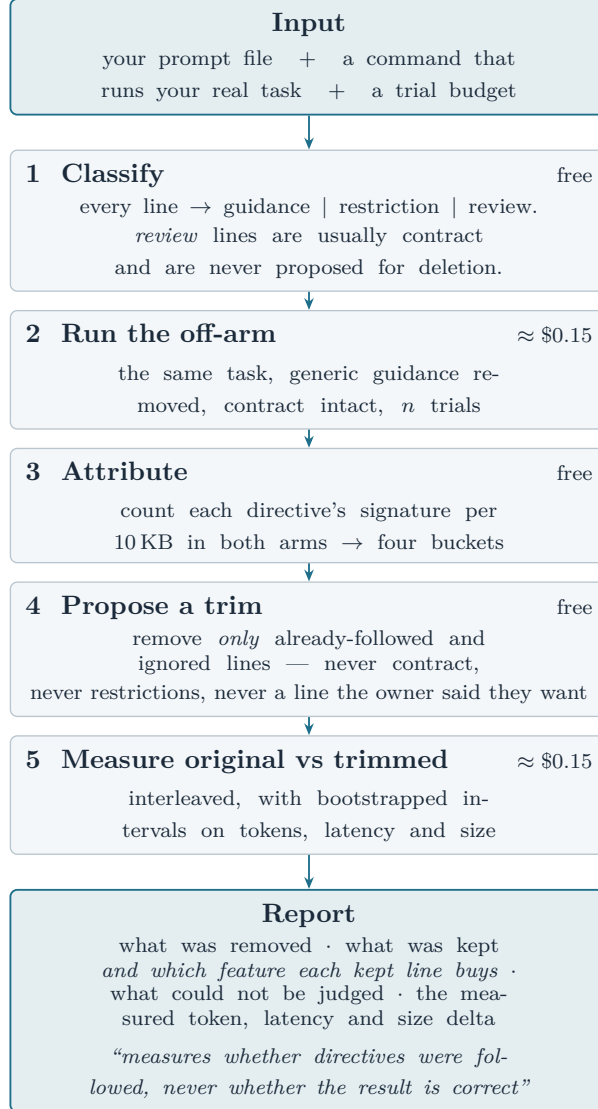


Figure 10: The released skill applies the same pipeline to a reader’s own prompt. It never deletes: lines surviving attribution are features being bought, and whether to keep buying them is the owner’s call, not the tool’s. Only the two marked steps call a model.

10 A validation protocol for behavioural graders

Every finding above rests on graders that drive generated pages like a user. Six defects were found in our own graders over the course of the study, and each produced a clean, plausible, wrong number. They motivate three controls that we now apply before any model output is judged, and that we recommend for any behavioural evaluation of generated code.

The three controls. A hand-written *correct* implementation of the task, which the grader must score 100%; an *empty* page, which it must score 0%; and a *single-rule mutation* of the correct implementation, which must fail exactly the checks that name that rule and no others. Together they establish that the grader recognises correct work, passes nothing by default, and observes each behaviour it claims to test.

What they would have caught. The table task scored exactly 8 of 10 in all 24 runs —

constant across two models and four levels, the signature of a broken check. Two checks were failing deterministically: one clicked the `<th>` while every generated file attached its sort handler to a `<button>` inside it, and events propagate up, not down; the other compared rows visible on page one, where a filter that reduces 24 records to 19 still shows 10. Corrected, all 24 files score 10 of 10. A rule checker double-counted every hex colour because `<style>` sits inside `<body>`, and scored compliant `:root` definitions as violations. A visible-text check read hidden nodes through `textContent`, inverting an early pilot. A missing element read as an empty string, indistinguishable from a valid field, so an empty page scored 6 of 31. And our own reference implementation parsed an ISO date as UTC midnight, shifting it a day earlier under local `getDate()` — the check that caught it was right and the reference was wrong, which is the direction a validated instrument is supposed to resolve. Each of the first four was found only by asking why a number was suspiciously stable; each of the last two was found by the controls at once.

11 Related work

A substantial literature optimises prompts automatically — DSPy (Khattab et al., 2023), GEPA (Agrawal et al., 2025), SPRIG (Zhang et al., 2024), SePO (Tao et al., 2026), SPEAR (Lu et al., 2026) and *p1* (Gao et al., 2026) search for wording that scores better, and MAS-PromptBench (Bai and Shi, 2026) asks when that helps in multi-agent settings. That work asks what the best prompt is; we ask what correct but unrequested content costs and buys, which optimisation presupposes.

The harm hypothesis has empirical precedent on earlier models: irrelevant context measurably distracts them (Shi et al., 2023), longer inputs alone degrade reasoning (Levy et al., 2024), and information in the middle of a long context is used less (Liu et al., 2023). Our ceiling result on current models is positioned against that precedent rather than in ignorance of it: the guidance here is relevant and correct, the tasks are short, and the project rules were placed first. Prompt compression (Jiang et al., 2023) is the standard remedy for input cost; our finding that the bill lands on the output, and that the output growth is requested features rather than verbosity, is the case compression does not address. Instruction-following benchmarks (Zhou et al., 2023) establish that current models follow checkable constraints reliably, which our compliance result corroborates; sensitivity to spurious prompt features (Sclar et al., 2023) is a related but distinct concern. Finally, the argument that agent evaluations should treat cost as a first-class metric alongside accuracy (Kapoor et al., 2024) is one this paper takes literally. Both vendors publish prescriptive prompting guidance (Anthropic, 2026; OpenAI, 2026), and skills accumulate general material of the kind measured here by imitation of it.

12 Data, code and reproducibility

All experiment code, raw run logs, behavioural graders, reference implementations and the verification script are released under the MIT licence; the paper text and figures under CC BY 4.0. Harness versions are pinned in `requirements.txt` (deepagents 0.6.12, langchain-openai 1.3.3, openai 2.44.0; Node 20 with jsdom 29). API nondeterminism means runs are not seed-reproducible; the logs of the runs reported here are included in full so every number can be re-derived offline.

Everything measured here is published, including the skill-scale artifact used for the highest dose point, which was composed for this study (`experiments/longskill/build_variants.py`) precisely so that no part of the record depends on material we are not free to release.

13 Limitations

Two sibling models from one vendor, one task family (self-contained front-end builds), $n = 3$ per cell in the ladder and $n = 1$ in the real-skill pilot, and a correctness instrument at ceiling whose minimum detectable effect is stated in §7. Reported output tokens include reasoning tokens; file size is the visible proxy. Latency in the ladder was not protected against run-order drift, so tokens are the primary cost measure. The zero-guidance arm still carries the agent harness's own base prompt. The ladder reaches 811 words; the skill-scale arm reaches 5,840 on a synthetic artifact of our own construction, which is representative in shape but is not a production skill in production use; the composition figures quoted for it in §5 are measured for the code-fence share and estimated for the remainder. Designs were fixed in code before their runs but not externally registered. The convergence question is open: a length-invariant metric with a positive-control arm, more than six runs per group, and a second vendor are the obvious next study.

14 What follows for practice

The defensible recommendation is short, and it is not "shorter is better."

Every generic line is a purchase order. The model will not apply it unprompted, will apply it when asked, and will bill for it: 42% more output tokens and 38% more latency at 811 words on a fixed task, twice the latency on an open-ended one, and close to nothing for several thousand words of templates and specification, because those describe rather than request.

Keep what you want to buy. Reduced-motion support, ARIA, responsive units and empty-state handling are real features with real users. If a team wants them, the guidance is worth its cost, and the cost is now known. If it does not, the same lines are pure overhead.

Restrictions are free; requests are billed. Rules that ask for restraint cost nothing. Rules that ask for work cost as much as guidance, however they are phrased.

A procedure. List each generic line in the skill. Run the task once at zero guidance and check mechanically which lines are already satisfied — the five-rule compliance test in §7 is that tool. Delete the satisfied ones. For each remaining line, decide whether its output is wanted. Re-measure output tokens. Repeat at the next model release, because the boundary between what a model does unprompted and what it does when asked moves with every one.

References

- Lakshya A Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziem, et al. GEPA: Reflective prompt evolution can outperform reinforcement learning. *arXiv preprint arXiv:2507.19457*, 2025.
- Anthropic. Prompting best practices. <https://platform.claude.com/docs/en/build-with-claude/prompt-engineering/claude-prompting-best-practices>, 2026. Harvested 2026-08-30; re-diffed 2026-09-04.
- Juyang Bai and Laixi Shi. MAS-PromptBench: When does prompt optimization improve multi-agent llm systems? *arXiv preprint arXiv:2606.23664*, 2026.
- Zhaolin Gao, Yu, Wang, Bo Liu, Thorsten Joachims, Kianté Brantley, and Wen Sun. p1: Better prompt optimization with fewer prompts. *arXiv preprint arXiv:2604.08801*, 2026.
- Huiqiang Jiang, Qianhui Wu, Chin-Yew Lin, et al. LLMingua: Compressing prompts for accelerated inference of large language models. *arXiv preprint arXiv:2310.05736*, 2023.

- Sayash Kapoor, Benedikt Stroebel, Zachary S. Siegel, et al. AI agents that matter. *arXiv preprint arXiv:2407.01502*, 2024.
- Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, et al. DSPy: Compiling declarative language model calls into self-improving pipelines. *arXiv preprint arXiv:2310.03714*, 2023.
- Mosh Levy, Alon Jacoby, and Yoav Goldberg. Same task, more tokens: the impact of input length on the reasoning performance of large language models. *arXiv preprint arXiv:2402.14848*, 2024.
- Nelson F. Liu, Kevin Lin, John Hewitt, et al. Lost in the middle: How language models use long contexts. *arXiv preprint arXiv:2307.03172*, 2023.
- Mengyin Lu, Cong Feng, Huimin Han, Guangming Lu, Yu Sun, Xiaonan Ding, Shihui Long, Fengyi Li, and Tanvi Motwani. SPEAR: Code-augmented agentic prompt optimization. *arXiv preprint arXiv:2605.26275*, 2026.
- OpenAI. Prompt engineering. <https://developers.openai.com/api/docs/guides/prompt-engineering>, 2026. Harvested 2026-08-30; re-diffed 2026-09-04.
- Melanie Sclar, Yejin Choi, Yulia Tsvetkov, et al. Quantifying language models’ sensitivity to spurious features in prompt design. *arXiv preprint arXiv:2310.11324*, 2023.
- Freda Shi, Xinyun Chen, Kanishka Misra, et al. Large language models can be easily distracted by irrelevant context. *arXiv preprint arXiv:2302.00093*, 2023.
- Wangcheng Tao, Han Wu, and Weng-Fai Wong. SePO: Self-evolving prompt agent for system prompt optimization. *arXiv preprint arXiv:2606.04465*, 2026.
- Lechen Zhang, Tolga Ergen, Lajanugen Logeswaran, Moontae Lee, and David Jurgens. SPRIG: Improving large language model performance by system prompt optimization. *arXiv preprint arXiv:2410.14826*, 2024.
- Jeffrey Zhou, Tianjian Lu, Swaroop Mishra, et al. Instruction-following evaluation for large language models. *arXiv preprint arXiv:2311.07911*, 2023.